

Regression Implementations in Python

Linear Regression (`linearReg.py`)

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.datasets import fetch_california_housing
5 from sklearn.metrics import mean_squared_error, r2_score
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.linear_model import LinearRegression
8
9 # =====
10 # Task 1: Load and preprocess the dataset
11 # =====
12
13 # Fetch the dataset and load it into a pandas DataFrame
14 california = fetch_california_housing()
15 df = pd.DataFrame(california.data, columns=california.feature_names)
16 df['Target'] = california.target
17
18 # Select the single feature: average number of rooms per dwelling ('AveRooms')
19 X_raw = df[['AveRooms']].values
20 y = df['Target'].values
21
22 # Preprocessing: Filter out extreme outliers in 'AveRooms' for a more meaningful
23 # linear fit
24 # (The dataset contains some estates with an unusually high number of rooms)
25 mask = X_raw[:, 0] < 10
26 X_filtered = X_raw[mask]
27 y_filtered = y[mask]
28
29 # Feature Scaling: Crucial for Gradient Descent to converge efficiently
30 scaler = StandardScaler()
31 X_scaled = scaler.fit_transform(X_filtered)
32
33 # Add a bias term (a column of ones,  $x_0 = 1$ ) to X for vectorized operations
34 m = len(X_scaled)
35 X_b = np.c_[np.ones((m, 1)), X_scaled]
36
37 # =====
38 # Task 2: Implement Linear Regression
39 # =====
40
41 # --- Method A: Normal Equation ---
42 # Using NumPy's linear algebra module for optimal memory performance
43 theta_normal = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y_filtered)
44 y_predict_normal = X_b.dot(theta_normal)
45
46 # --- Method B: Gradient Descent ---
47 learning_rate = 0.1
48 n_iterations = 1000
49
50 # Random initialization of weights
```

```

50 np.random.seed(42)
51 theta_gd = np.random.randn(2)
52
53 for iteration in range(n_iterations):
54     # Calculate gradients
55     gradients = 2/m * X_b.T.dot(X_b.dot(theta_gd) - y_filtered)
56     # Update weights
57     theta_gd = theta_gd - learning_rate * gradients
58
59 y_predict_gd = X_b.dot(theta_gd)
60
61 # =====
62 # Task 3: Evaluate model performance
63 # =====
64
65 print("--- Normal Equation Performance ---")
66 print(f"MSE: {mean_squared_error(y_filtered, y_predict_normal):.4f}")
67 print(f"R-squared: {r2_score(y_filtered, y_predict_normal):.4f}\n")
68
69 print("--- Gradient Descent Performance ---")
70 print(f"MSE: {mean_squared_error(y_filtered, y_predict_gd):.4f}")
71 print(f"R-squared: {r2_score(y_filtered, y_predict_gd):.4f}\n")
72
73 print(f"Theta (Normal): {theta_normal}")
74 print(f"Theta (GD): {theta_gd}")
75
76
77 # SKLearn Implementation for comparison
78 from sklearn.model_selection import train_test_split
79 from sklearn.linear_model import LinearRegression
80 # =====
81 # Task 1: Load and preprocess the dataset
82 # =====
83
84 # Fetch data and extract 'AveRooms' (Index 2) and target
85 california = fetch_california_housing()
86 X_raw = california.data[:, 2].reshape(-1, 1)
87 y = california.target
88
89 # Filter outliers to match previous visualization
90 mask = X_raw[:, 0] < 10
91 X_filtered = X_raw[mask]
92 y_filtered = y[mask]
93
94 # Feature Scaling
95 scaler = StandardScaler()
96 X_scaled = scaler.fit_transform(X_filtered)
97
98 # =====
99 # Task 2: Implement Scikit-Learn Linear Regression
100 # =====
101
102 # Instantiate and fit the model in two lines
103 model = LinearRegression()
104 model.fit(X_scaled, y_filtered)
105
106 # Generate predictions
107 y_predict_sklearn = model.predict(X_scaled)
108
109 # =====
110 # Task 3: Evaluate model performance
111 # =====
112

```

```

113 print("--- Scikit-Learn Linear Regression Performance ---")
114 print(f"MSE: {mean_squared_error(y_filtered, y_predict_sklearn):.4f}")
115 print(f"R-squared: {r2_score(y_filtered, y_predict_sklearn):.4f}")
116 print(f"Intercept (Bias): {model.intercept_:.4f}")
117 print(f"Coefficient (Weight): {model.coef_[0]:.4f}\n")
118
119 # =====
120 # Task 4: Visualize the fitted line
121 # =====
122
123 # Scatter plot of the original data points
124 plt.scatter(X_scaled, y_filtered, alpha=0.2, color='blue', label='Data points',
125             s=10)
126
127 # Plot the regression line (both will overlap if they converged correctly)
128 plt.plot(X_scaled, y_predict_normal, color='red', linewidth=2, label='Fitted Line
129         (Normal Eq)')
130 plt.plot(X_scaled, y_predict_gd, color='yellow', linestyle='--', linewidth=2,
131         label='Fitted Line (Gradient Descent)')
132 plt.plot(X_scaled, y_predict_sklearn, color='green', linewidth=2, label='Fitted
133         Line (sklearn)')
134
135 plt.title('Linear Regression: California Housing (AveRooms vs Price)')
136 plt.xlabel('Average Rooms per Dwelling (Standardized)')
137 plt.ylabel('Median House Value (in $100,000s)')
138 plt.legend()
139 plt.grid(True, alpha=0.3)
140 plt.savefig('expl.png')

```

Polynomial Regression (polynomialReg.py)

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.metrics import mean_squared_error, r2_score
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.linear_model import LinearRegression
8
9 # =====
10 # Task 1: Load and preprocess the dataset
11 # =====
12
13 # Load the Auto MPG dataset
14 df = sns.load_dataset('mpg')
15
16 # Drop missing values
17 df = df.dropna(subset=['mpg', 'displacement'])
18
19 # Extract feature (X) and target (y)
20 X_raw = df[['displacement']].values
21 y = df['mpg'].values
22
23 # Create Polynomial Features from scratch (Degree 2)
24 # We add x^2 as a new feature column next to the original x
25 X_poly_raw = np.c_[X_raw, X_raw**2]
26
27 # Feature Scaling: Absolutely critical for Gradient Descent to converge
28 # when dealing with polynomial features (like displacement^2 which gets very large)
29 scaler = StandardScaler()
30 X_poly_scaled = scaler.fit_transform(X_poly_raw)
31
32 # Add a bias term (a column of ones, x0 = 1) to X for vectorized operations
33 m = len(X_poly_scaled)
34 X_b = np.c_[np.ones((m, 1)), X_poly_scaled]
35
36 # =====
37 # Task 2: Implement Polynomial Regression
38 # =====
39
40 # --- Method A: Normal Equation ---
41 # Mathematical optimization using:  $\theta = (X^T * X)^{-1} * X^T * y$ 
42 theta_normal = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
43 y_predict_normal = X_b.dot(theta_normal)
44
45 # --- Method B: Gradient Descent ---
46 learning_rate = 0.1
47 n_iterations = 1000
48
49 # Random initialization of weights (3 weights: bias, x, x^2)
50 np.random.seed(42)
51 theta_gd = np.random.randn(3)
52
53 for iteration in range(n_iterations):
54     # Calculate gradients
55     gradients = 2/m * X_b.T.dot(X_b.dot(theta_gd) - y)
56     # Update weights
57     theta_gd = theta_gd - learning_rate * gradients
58
59 y_predict_gd = X_b.dot(theta_gd)
60
61 # =====
```

```

62 # Task 3: Scikit-Learn Implementation
63 # =====
64
65 # We pass the same scaled polynomial features to sklearn for a direct comparison
66 model = LinearRegression()
67 model.fit(X_poly_scaled, y)
68 y_predict_sklearn = model.predict(X_poly_scaled)
69
70 # =====
71 # Task 4: Evaluate model performance
72 # =====
73
74 print("--- Normal Equation Performance ---")
75 print(f"MSE: {mean_squared_error(y, y_predict_normal):.4f}")
76 print(f"R-squared: {r2_score(y, y_predict_normal):.4f}\n")
77
78 print("--- Gradient Descent Performance ---")
79 print(f"MSE: {mean_squared_error(y, y_predict_gd):.4f}")
80 print(f"R-squared: {r2_score(y, y_predict_gd):.4f}\n")
81
82 print("--- Scikit-Learn Performance ---")
83 print(f"MSE: {mean_squared_error(y, y_predict_sklearn):.4f}")
84 print(f"R-squared: {r2_score(y, y_predict_sklearn):.4f}\n")
85
86 print(f"Theta (Normal): {theta_normal}")
87 print(f"Theta (GD): {theta_gd}")
88 print(f"Theta (Sklearn): [{model.intercept_} {model.coef_[0]} {model.coef_[1]}]")
89
90 # =====
91 # Task 5: Visualize the fitted curves
92 # =====
93
94 # To plot a smooth curve, we must sort the X values and their corresponding
   predictions
95 sort_index = np.argsort(X_raw[:, 0])
96 X_plot = X_raw[sort_index]
97 y_plot_normal = y_predict_normal[sort_index]
98 y_plot_gd = y_predict_gd[sort_index]
99 y_plot_sklearn = y_predict_sklearn[sort_index]
100
101 plt.figure(figsize=(10, 6))
102
103 # Scatter plot of the original data points
104 plt.scatter(X_raw, y, alpha=0.4, color='gray', label='Actual Data', s=20)
105
106 # Plot the regression curves
107 # We use thicker lines and varying styles so overlapping curves are visible
108 plt.plot(X_plot, y_plot_normal, color='red', linewidth=4, label='Fitted Curve
   (Normal Eq)')
109 plt.plot(X_plot, y_plot_gd, color='yellow', linestyle='--', linewidth=3,
   label='Fitted Curve (Gradient Descent)')
110 plt.plot(X_plot, y_plot_sklearn, color='black', linestyle=':', linewidth=2,
   label='Fitted Curve (sklearn)')
111
112 plt.title('Polynomial Regression (Degree 2): MPG vs. Displacement')
113 plt.xlabel('Engine Displacement (cu. inches)')
114 plt.ylabel('Miles Per Gallon (MPG)')
115 plt.legend()
116 plt.grid(True, alpha=0.3)
117 plt.savefig('exp2.png')

```